Rapport de stage : Mise en place d'une Modern Data Stack

SOMMAIRE :

I. Présentation du stage et du contexte 2

A. Présentation du stage 2

B. Présentation de l'entreprise 2

II. Présentation du projet 3

A. Présentation du projet technique 3

B. Les sources de données 4

1. Les flux de données (flow) 5

2. La base PostgreSQL et la transformation via DBT 8

3. Metabase et la création des tableaux de bord 9

III. Présentation de la méthode de travail (Méthode Agile Scrum) 12

IV. Chronologie de mes missions 13

A. Sprint 1 : Étude, Documentation et Mise en place technique 14

B. Sprint 2 : Mise en place des flux de données et stockage 15

C. Sprint 3 : Finalisation des flux et début du dashboarding 16

V. Conclusion et perspectives 18

I) Présentation du stage et du contexte : (Entreprise et déroulement)

A) Présentation du stage :

Le stage que j'ai effectué durant ma seconde année pour le BTS SIO option SLAM, s'est déroulé du 12 janvier 2026 au 20 février 2026 au sein de l'entreprise Alturing située à Paris dans le 14e arrondissement. Ce stage m'a permis d'enrichir mes connaissances et mes compétences et aussi de les mettre en pratique.

B) Présenation de l'entreprise :

Alturing est une Entreprise de Services du Numérique (ESN) filiale du Groupe La Poste. Elle opère spécifiquement au sein de la branche GeoPost (qui regroupe les activités de colis express comme Chronopost ou DPD).

Créer en 1996, Alturing fonctionne comme une **DSI** (Direction des Systèmes d'Information) pour ses clients internes. Son siège social est situé à Paris, mais elle dispose d'un pôle opérationnel et technique majeur près d'Angers.

La mission principale d'Alturing est d'assurer la traçabilité de bout en bout des flux logistiques. Dans un contexte où tout le monde utilise la livraison de colis ou encore avec l'explosion du e-commerce, l'entreprise garantit la disponibilité et l'intégrité des informations liées aux colis.

Ses activités se divisent en deux axes majeurs pour un informaticien :

La première de ces missions est l'exploitation et le Maintien en Condition Opérationnelle (MCO) des systèmes d'information. Cette activité se déroule dans le site situé à Paris, plus précisément dans le 14e arrondissements.

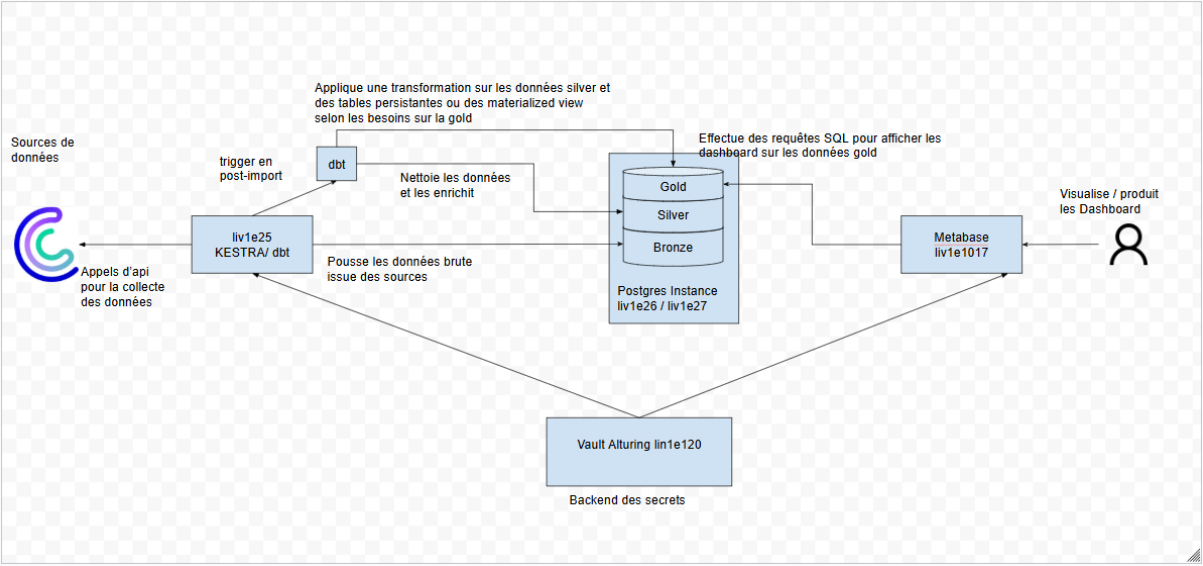
La seconde est la supervision qui consiste à superviser 24h/24 des flux informatiques. Puisqu'une indisponibilité de service peut paralyser les centres de tri nationaux, Alturing est garante du Maintien en Condition Opérationnelle des systèmes de Chronopost. Cette activité se déroule dans le site situé à Angers.

II) Présentation du projet :

A) Présentation du projet technique :

Mon stage au sein d'Alturing a eu pour mission principale la création et l'automatisation d'une **Modern Data Stack**. L'objectif technique était de concevoir des flux automatisés capables de récupérer des données brutes de supervision, de les stocker de manière sécurisée, puis de les transformer pour les rendre exploitables visuellement.

Pour comprendre la logique de ce projet, voici l'architecture globale du système mis en place :



Ce schéma explique le fonctionnement de notre projet qui est un processus où chaque outil a un rôle précis. Tout commence par la récupération des données à la source : nous nous connectons aux API de nos différents logiciels pour y piocher les informations brutes. Pour automatiser tout cela, nous utilisons Kestra, qui agit comme le chef d'orchestre du projet. C'est cet outil qui se charge de lancer les collectes de données à intervalles réguliers. Pour que cela se fasse en toute sécurité, Kestra ne stocke aucun mot de passe dans son code : il va les récupérer au dernier moment dans le Vault Alturing, qui fonctionne comme un coffre-fort numérique.

Une fois les informations récupérées par Kestra, elles sont envoyées et stockées dans une base de données PostgreSQL. C'est à ce moment-là que l'outil dbt (Data Build Tool) entre en piste. Son travail est de faire le ménage : il trie, nettoie et organise toutes ces

données brutes pour les rendre exploitables. Enfin, le parcours se termine avec Metabase. C'est l'outil d'affichage. Il vient lire les données propres directement dans la base PostgreSQL pour les transformer en graphiques et en tableaux de bord clairs, que les utilisateurs puissent comprendre d'un simple coup d'œil.

B) Les sources de données :

Pour que nos tableaux de bord soient vraiment utiles et reflètent l'état réel de nos systèmes, nous devions connecter notre architecture à plusieurs sources d'informations. Nous récupérons par exemple des données depuis **GLPI**, notre outil de gestion de parc informatique et de tickets d'incidents, mais aussi depuis diverses applications spécifiques à la logistique du groupe. L'idée de base de notre plateforme était d'être assez généraliste pour pouvoir se brancher sur n'importe quel logiciel interne.

Cependant, dans le cadre précis de mon stage, mon travail s'est concentré presque exclusivement sur une source centrale : Centreon. C'est le système de supervision infrastructure et réseau. Il surveille en permanence l'état des serveurs, des bases de données et des applications.

Centreon a vraiment été le point de départ de toutes mes missions. J'ai dû étudier en détail le fonctionnement de son API via sa documentation pour comprendre comment en extraire toutes les configurations techniques (comme la liste des hôtes, les groupes d'hôtes ou encore les times périodes). C'est à partir des données de Centreon que j'ai pu construire toute la chaîne de travail. J'ai d'abord créé les flux d'extraction automatisés dans Kestra, j'ai ensuite modélisé et généré les tables dans PostgreSQL pour stocker ces données proprement, et pour finir, j'ai exploité ces bases pour créer des tableaux de bord de suivi dans Metabase. En résumé, les données de Centreon ont été la source première du début à la fin de la chaîne.

1) <u>Les flux de données (flow) :</u>

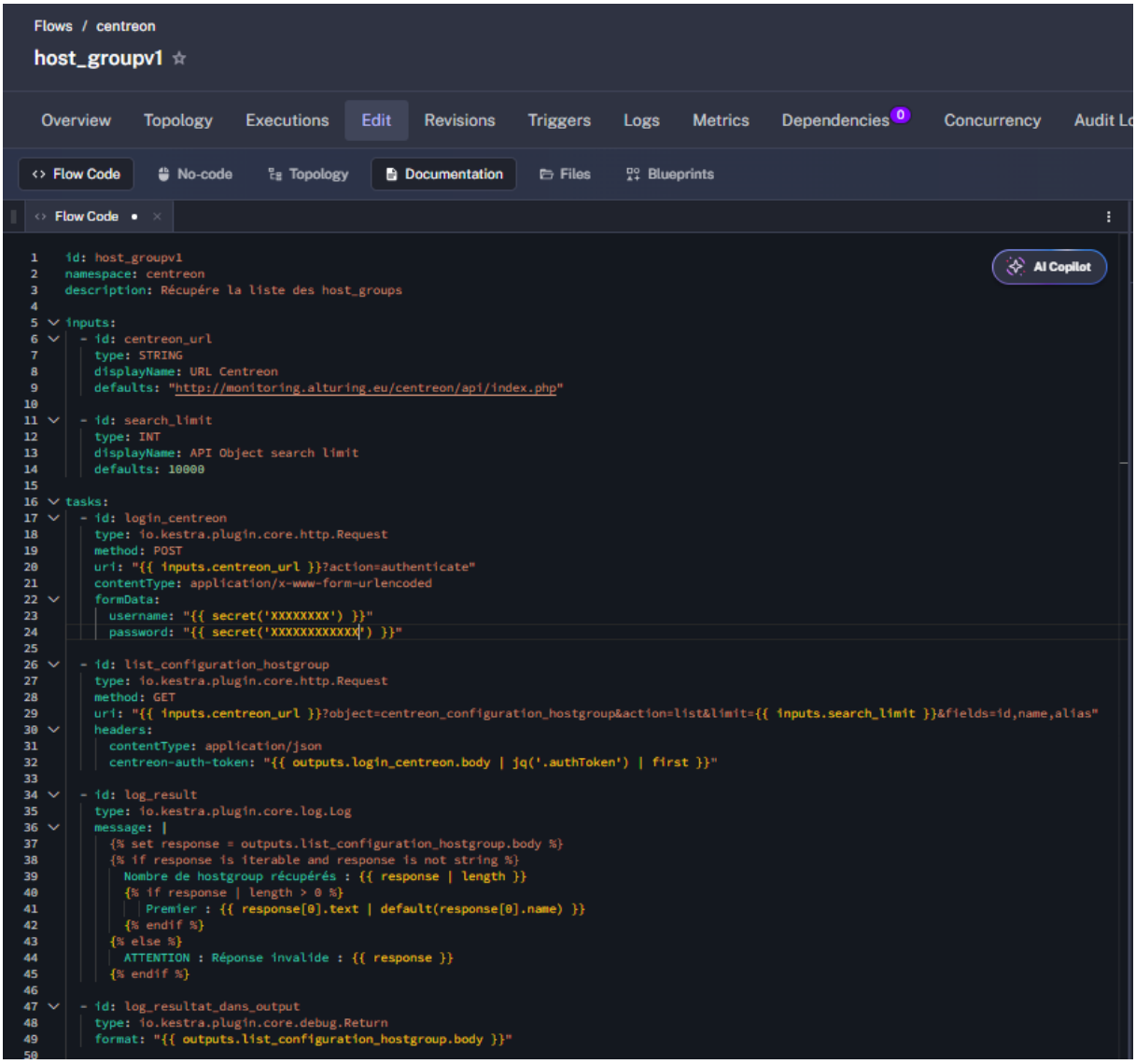
Au cours de mon stage, j'ai eu l'opportunité de créer et de configurer de multiples flux dans l'orchestrateur Kestra. L'objectif était de couvrir un périmètre large de la configuration de Centreon. J'ai ainsi mis en place des pipelines automatisés pour extraire les **Groupes d'hôtes** (Host Groups), les **Périodes** (Time Periods), mais aussi les hôtes, les services et les modèles (Templates). Bien que chaque flux possède ses propres spécificités liées à l'API, ils partagent tous une logique d'architecture commune

Pour réaliser ce projet, ma principale mission a été de créer et de configurer plusieurs flux distincts dans l'orchestrateur Kestra afin de couvrir un large périmètre des éléments de configuration de Centreon. Comme l'illustre la capture de mon espace de travail, j'ai conçu et automatisé divers pipelines spécifiques. J'ai notamment développé le flux **host_groupv1** pour l'extraction globale des groupes d'hôtes, ainsi que les flux **templates_hotesv1** et **service_templates_v1** pour récupérer les différents modèles appliqués aux machines et à leurs services. Pour compléter cette collecte, j'ai également mis en place le flux **timesperiod_templatev1** chargé de remonter les périodes de temps définies dans la supervision, et le flux **host_service-url_action** pour lier les actions spécifiques aux URL associées aux services. Bien que chaque flux possède ses propres contraintes techniques, comme des URL d'API ou des champs JSON différents selon l'objet ciblé, ils partagent tous une architecture et une logique de traitement communes. Pour bien comprendre le fonctionnement de cette logique, voici l'analyse détaillée d'un flux de bout en bout, en prenant l'exemple précis de l'extraction **host_groupv1**.

Ce code permet de récupérer, transformer et sauvegarder automatiquement des données de configuration de groupes d'hôtes (host groups) vers une base de données. Avant de s'exécuter, le flux s'appuie sur des paramètres d'entrée (inputs) définissant par exemple l'URL de l'API ou la limite de recherche. Le script débute véritablement par la tâche « login_centreon », qui s'authentifie de manière sécurisée pour autoriser l'accès. Immédiatement après, la tâche « list_configuration_hostgroup » interroge l'API pour télécharger la liste brute des groupes d'hôtes. Des étapes de contrôle, comme « log_result », interviennent alors pour vérifier que la réponse est valide et compter le nombre d'éléments récupérés. Ensuite, la tâche « init_database » se connectE à une base PostgreSQL pour y créer la table centreon_table_host_groupv1 si elle n'existe pas encore. Puis, la tâche « merge_ndjson » a pour mission d'organiser proprement les

données brutes en extrayant uniquement l'identifiant (id) et le nom (alias) de chaque groupe, tout en y ajoutant la date d'exécution. Après ce formatage, les tâches « to_ion » puis « to_csv » convertissent ces informations en un fichier CSV structuré. Pour finir, la tâche « copy_to_db » prend le relais en important automatiquement tout le contenu de ce fichier CSV directement dans la table PostgreSQL préparée précédemment.

Partie 1 du code :




```yaml
id: host_groupv1
namespace: centreon
description: Récupère la liste des host_groups

inputs:
  - id: centreon_url
    type: STRING
    displayName: URL Centreon
    defaults: "http://monitoring.alturing.eu/centreon/api/index.php"

  - id: search_limit
    type: INT
    displayName: API Object search limit
    defaults: 10000

tasks:
  - id: login_centreon
    type: io.kestra.plugin.core.http.Request
    method: POST
    uri: "{{ inputs.centreon_url }}?action=authenticate"
    contentType: application/x-www-form-urlencoded
    formData:
      username: "{{ secret('XXXXXXXX') }}"
      password: "{{ secret('XXXXXXXXXXXX') }}"

  - id: list_configuration_hostgroup
    type: io.kestra.plugin.core.http.Request
    method: GET
    uri: "{{ inputs.centreon_url }}?object=centreon_configuration_hostgroup&action=list&limit={{ inputs.search_limit }}&fields=id,name,alias"
    headers:
      contentType: application/json
      centreon-auth-token: "{{ outputs.login_centreon.body | jq('.authToken') | first }}"

  - id: log_result
    type: io.kestra.plugin.core.log.Log
    message: |
      {% set response = outputs.list_configuration_hostgroup.body %}
      {% if response is iterable and response is not string %}
        Nombre de hostgroup récupérés : {{ response | length }}
        {% if response | length > 0 %}
          Premier : {{ response[0].text | default(response[0].name) }}
        {% endif %}
      {% else %}
        ATTENTION : Réponse invalide : {{ response }}
      {% endif %}

  - id: log_resultat_dans_output
    type: io.kestra.plugin.core.debug.Return
    format: "{{ outputs.list_configuration_hostgroup.body }}"
```

Partie 2 du code :

```yaml
- id: log_resultat_dans_output ···

- id: init_database
  type: io.kestra.plugin.jdbc.postgresql.Queries
  url: jdbc:postgresql://10.36.63.4:5432/test_centreon
  username: kestra_test
  password: 7vNz+uyeiDcVDQKo
  sql: |
    CREATE TABLE IF NOT EXISTS centreon_table_host_groupv1 (
        id SERIAL PRIMARY KEY,
        name VARCHAR(255),
        alias VARCHAR(255),
        host_group_id INT,
        log_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

- id: merge_ndjson
  type: io.kestra.plugin.core.storage.Write
  extension: .jsonl
  content: |
    {%- set response = json(outputs.list_configuration_hostgroup.body) -%}
    {%- if response.items is defined -%}
    {%- for item in response.items -%}
    {{ {"id": item.id, "name": item.text, "alias": item.text, "log_date": execution.startDate} | json }}
    {% endfor -%}
    {%- endif -%}

- id: to_ion
  type: io.kestra.plugin.serdes.json.JsonToIon
  from: "{{ outputs.merge_ndjson.uri }}"
  newLine: true

- id: to_csv
  type: io.kestra.plugin.serdes.csv.IonToCsv
  from: "{{ outputs.to_ion.uri }}"
  header: true
  fieldSeparator: ","
  textDelimiter: "\""

- id: copy_to_db
  type: io.kestra.plugin.jdbc.postgresql.CopyIn
  url: jdbc:postgresql://10.36.63.4:5432/test_centreon
  username: kestra_test
  password: 7vNz+uyeiDcVDQKo
  from: "{{ outputs.to_csv.uri }}"
  table: centreon_table_host_groupv1
  format: CSV
  header: true
  delimiter: ","
  quote: "\""
  columns:
    - name
    - alias
    - host_group_id
    - log_date
```

Le résultat s'affiche dans l'onglet « Outputs » de Kestra une fois que le flux a fini de tourner. C'est la preuve que tout a bien fonctionné. C'est grâce au passage en format CSV que les informations apparaissent sous forme de tableau, ce qui est bien plus pratique à lire que du code brut. Même si on regarde cet aperçu directement dans l'interface de Kestra, il montre exactement ce qui est envoyé dans la base PostgreSQL. Tout est bien rangé dans des colonnes claires : on y retrouve le nom, l'identifiant unique et la date précise de l'extraction :

name	alias	id	log_date
All	All	1	2026-02-09T13:27:23.731666Z
ALT	ALT	2	2026-02-09T13:27:23.731666Z
ALT-Part-IS-senonches	ALT-Part-IS-senonches	747	2026-02-09T13:27:23.731666Z
ALT-Partenaire-IS	ALT-Partenaire-IS	3	2026-02-09T13:27:23.731666Z
ALT-SC-CSV-Angers	ALT-SC-CSV-Angers	4	2026-02-09T13:27:23.731666Z
ALT-SC-ESX-Angers	ALT-SC-ESX-Angers	5	2026-02-09T13:27:23.731666Z
ALT-Siege	ALT-Siege	6	2026-02-09T13:27:23.731666Z
ALT-SYSTEM-ESX	ALT-SYSTEM-ESX	7	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-Epo	ALT-ZAPPLI-Epo	8	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-FootPrint	ALT-ZAPPLI-FootPrint	9	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-GLPI	ALT-ZAPPLI-GLPI	10	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-OPENPROJECT	ALT-ZAPPLI-OPENPROJECT	758	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-PortailCSV	ALT-ZAPPLI-PortailCSV	11	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-Qlikview	ALT-ZAPPLI-Qlikview	12	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-QualityC	ALT-ZAPPLI-QualityC	13	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-URL-DPD	ALT-ZAPPLI-URL-DPD	14	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-Vault	ALT-ZAPPLI-Vault	742	2026-02-09T13:27:23.731666Z

2) La base PostGreSQL et la transformation via DBT :

Une fois extraites par Kestra, les données sont stockées dans notre base de données PostgreSQL. La création des tables qui reçoivent ces informations se fait automatiquement au début de chaque flux grâce à une commande SQL préparant les colonnes nécessaires. Lors de cette étape, j'ai configuré des index sur les colonnes contenant les identifiants. Concrètement, un index est une technique d'optimisation qui permet à la base de données de trouver instantanément une ligne spécifique sans avoir à lire tout le tableau. C'est indispensable pour que les futurs tableaux de bord s'affichent rapidement, sans faire patienter l'utilisateur lors du chargement des graphiques.

Pour garder notre base de données bien rangée et éviter de mélanger des données de niveaux différents, nous trions les informations dans trois espaces distincts : Bronze, Silver et Gold. Le premier espace, la couche Bronze, reçoit les données brutes exactement comme elles sortent des fichiers CSV de Kestra. Ensuite, nous utilisons l'outil dbt (Data Build Tool), qui sert à automatiser la transformation des données grâce

à des requêtes SQL. L'outil dbt récupère les données de la couche Bronze, retire les doublons, corrige les erreurs de format et rassemble les informations utiles.

Ce résultat propre et vérifié est alors sauvegardé dans la couche Silver. Pour terminer, dbt applique de dernières opérations de regroupement pour créer la couche Gold. Cette ultime couche contient des tableaux très ciblés et simplifiés, spécialement préparés pour être lus facilement par notre outil de création de graphiques.

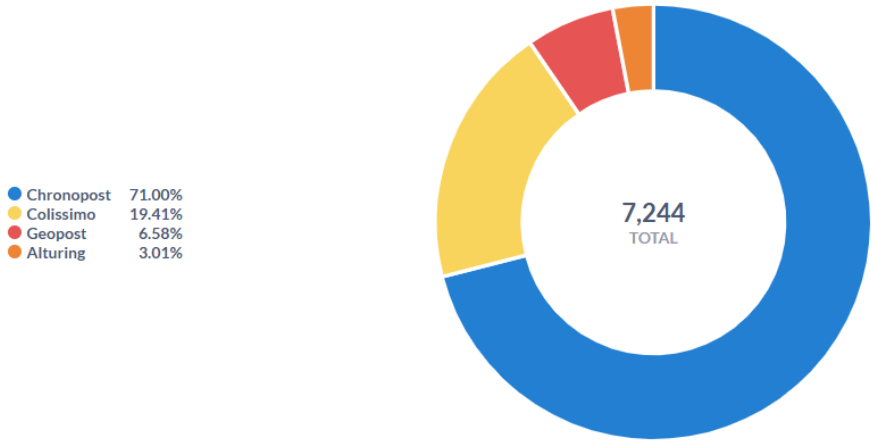
3) Metabase et la création des tableaux de bord :

Pour la toute dernière étape, j'ai utilisé Metabase, notre outil pour créer des graphiques et des tableaux de bord. Si nous regardons le schéma de départ, nous avions prévu d'utiliser l'outil dbt pour nettoyer automatiquement les données dans notre base. Mais comme ce grand projet vient tout juste de démarrer chez Alturing, nous n'avons pas eu le temps de mettre cette partie en place pendant les quelques semaines de mon stage. Pour avoir quand même un résultat concret et utile à présenter, nous avons décidé de brancher Metabase directement sur notre base PostgreSQL. L'idée, c'était de livrer une première version qui fonctionne bien, pour que l'entreprise ait un bon point de départ à améliorer plus tard.

Comme nous n'avions pas dbt pour faire le tri, j'ai écrit directement les requêtes SQL dans Metabase. Cela nous a permis de filtrer les données qui arrivaient de Centreon, de les regrouper et de préparer nos premiers affichages. Pour vous donner un exemple, voici le code que j'ai conçu pour générer le graphique de répartition des machines par Business Unit (BU) :

Cette requête SQL a pour but de classer et de compter les serveurs (les hôtes) en fonction de la Business Unit (BU) à laquelle ils appartiennent. Pour automatiser ce tri, la requête s'appuie sur une condition CASE qui analyse le nom de l'instance, représenté par la colonne instance_name. Par exemple, si le nom de l'instance contient la chaîne de caractères "liv", la machine est automatiquement étiquetée comme appartenant à "Alturing". L'utilisation de l'opérateur ILIKE est très pratique ici, car il permet de chercher ces lettres n'importe où dans le nom, sans se soucier des majuscules ou des minuscules. La requête applique cette même logique de mots-clés pour identifier les serveurs de "Geopost", "Colissimo" et "Chronopost", tandis que tous les autres serveurs non identifiés sont regroupés dans une catégorie "Autre". Une fois ce tri effectué, la commande count(DISTINCT host_id) intervient pour compter le nombre de machines uniques dans chaque catégorie, garantissant ainsi qu'un même serveur ne soit pas compté plusieurs fois. Enfin, l'instruction GROUP BY 1 rassemble le tout pour afficher le résultat final regroupé par les noms de BU que nous venons de créer. C'est typiquement cette requête qui permet ensuite à Metabase de générer un graphique clair, comme un diagramme circulaire ou un graphique en barres.

Resultat sous la forme d'un graphique camembert :



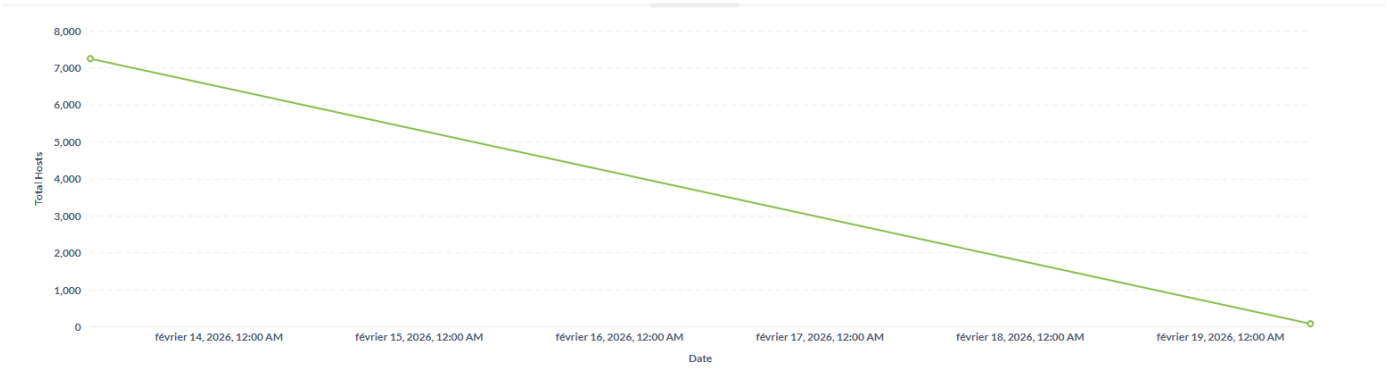
Pour les hôtes supervisés, cette requête sert à observer comment leur nombre évolue dans le temps. Elle utilise la fonction date_trunc pour découper et regrouper les données minute par minute en se basant sur la date d'enregistrement (log_date). Ensuite, on compte le nombre exact de serveurs présents à chaque instant grâce à la commande count(DISTINCT host_id). L'utilisation du mot DISTINCT est très importante ici, car elle garantit qu'une même machine ne sera pas comptée deux fois. Pour finir, les instructions GROUP BY et ORDER BY 1 ASC permettent d'organiser tous ces résultats dans un ordre chronologique. Concrètement, dans Metabase, cette requête est idéale pour tracer un graphique en forme de courbe temporelle, ce qui permet aux équipes de voir d'un simple coup d'œil si de nouvelles machines ont été ajoutées ou retirées de la supervision au fil du temps.

Évolution des hôtes supervisés

test_centreon

```sql
SELECT
    date_trunc('minute', log_date) AS "Date",
    count(DISTINCT host_id) AS "Total Hosts"
FROM centreon_host_service_table
GROUP BY 1
ORDER BY 1 ASC
```

Resultat sous la forme d’un graphique évolutif :



III) Présentation de la méthode de travail (Méthode Agile Scrum)

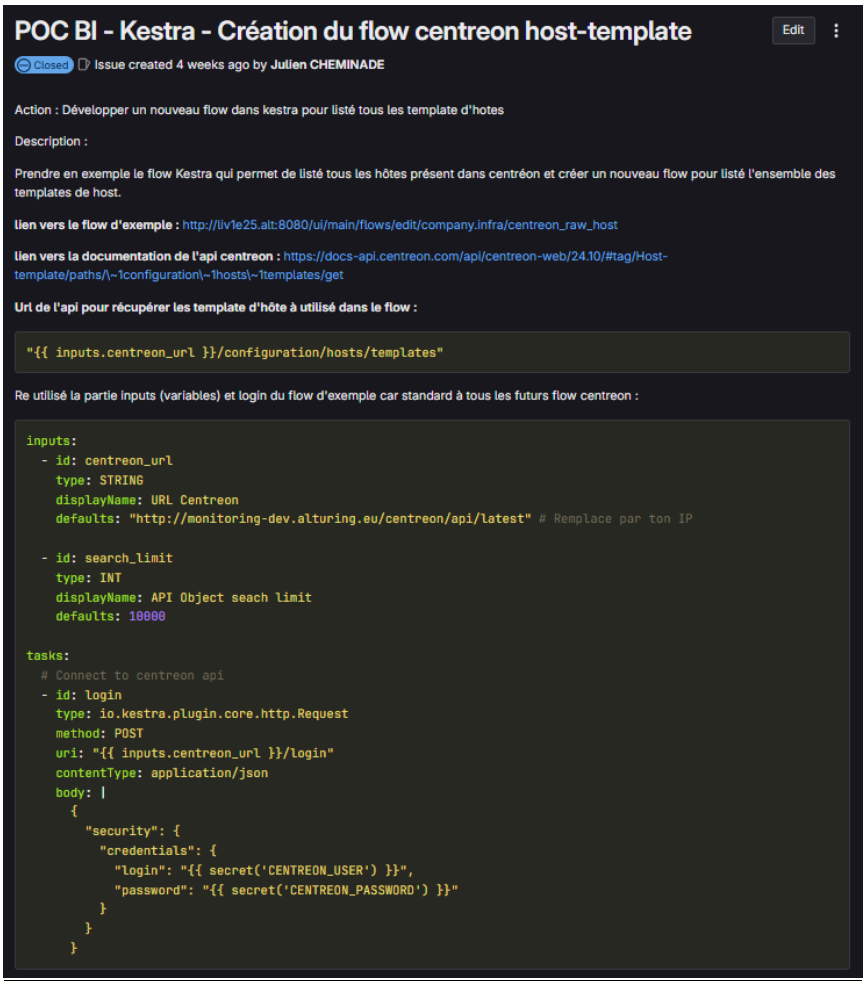
Pour mener à bien ce projet, nous avons travaillé selon la méthode Agile Scrum, en utilisant GitLab pour suivre nos tickets (les « Issues »). Cette méthode s'est traduite par découper le travail en périodes de deux semaines appelées « Sprints », on se concentre sur une mission précise à chaque fois, comme créer un flow ou ajouter un schéma SQL. À la fin de chaque cycle, on fait le point lors d'une réunion pour valider ce qui a été fait et préparer la suite. Les réunions avaient lieu le Lundi à 16h30 tous les deux semaines. Les 3 différents sprints ont été :

Sprint 1 : Étude du besoin, documentation sur les outils d'orchestration et mise en place de l'environnement technique.

Sprint 2 : Étude approfondie de la documentation de l'API Centreon et de Kestra, puis démarrage de la mise en place des flux de données.

Sprint 3 : Finalisation des différents flux d'extraction, création des tables PostgreSQL, et début du travail sur le dashboarding dans Metabase.

Exemple détaillé d'une « Issue » sur GitLab :



POC BI - Kestra - Création du flow centreon host-template

Edit

Closed · Issue created 4 weeks ago by **Julien CHEMINADE**

Action : Développer un nouveau flow dans kestra pour listé tous les template d'hotes

Description :

Prendre en exemple le flow Kestra qui permet de listé tous les hôtes présent dans centréon et créer un nouveau flow pour listé l'ensemble des templates de host.

lien vers le flow d'exemple : http://liv1e25.alt:8080/ui/main/flows/edit/company.infra/centreon_raw_host

lien vers la documentation de l'api centreon : https://docs-api.centreon.com/api/centreon-web/24.10/#tag/Host-template/paths/\~1configuration\~1hosts\~1templates/get

Url de l'api pour récupérer les template d'hôte à utilisé dans le flow :

```
"{{ inputs.centreon_url }}/configuration/hosts/templates"
```

Re utilisé la partie inputs (variables) et login du flow d'exemple car standard à tous les futurs flow centreon :

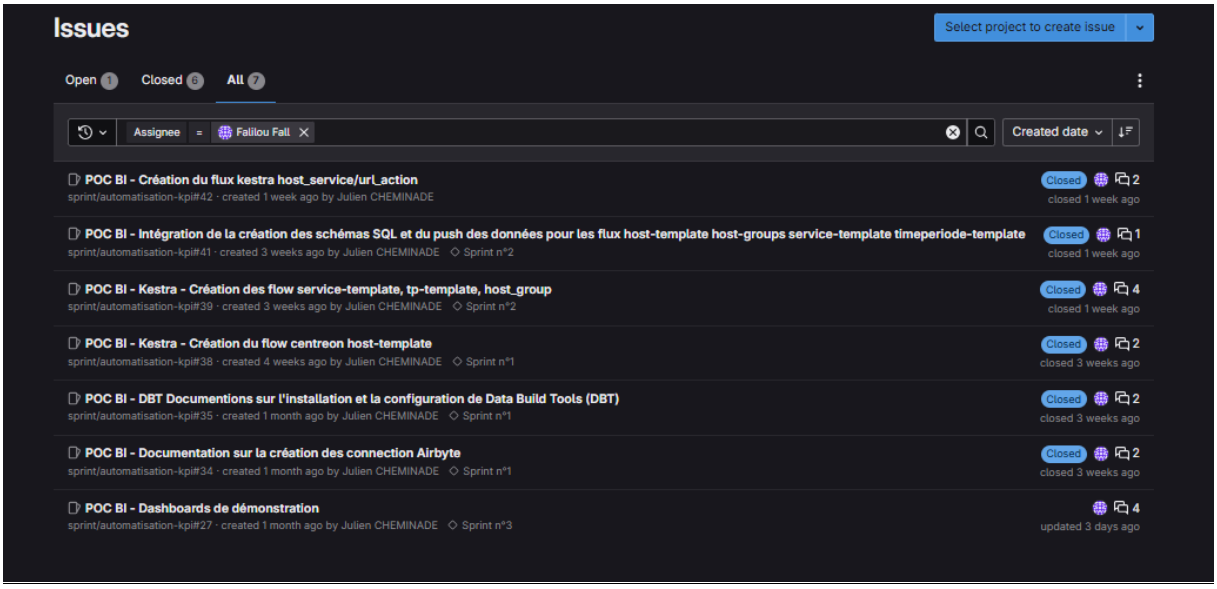
```
inputs:
  - id: centreon_url
    type: STRING
    displayName: URL Centreon
    defaults: "http://monitoring-dev.alturing.eu/centreon/api/latest" # Remplace par ton IP

  - id: search_limit
    type: INT
    displayName: API Object seach limit
    defaults: 10000

tasks:
  # Connect to centreon api
  - id: login
    type: io.kestra.plugin.core.http.Request
    method: POST
    uri: "{{ inputs.centreon_url }}/login"
    contentType: application/json
    body: |
      {
        "security": {
          "credentials": {
            "login": "{{ secret('CENTREON_USER') }}",
            "password": "{{ secret('CENTREON_PASSWORD') }}"
          }
        }
      }
```

IV) Chronologie de mes missions

Suivi de mes missions sur GitLab réparties par Sprints :



Pour construire cette Modern Data Stack, mes missions n'ont pas été une simple suite de tâches indépendantes. Comme l'illustre le schéma d'architecture du projet, mon travail a suivi le parcours exact de la donnée, de sa collecte à sa restitution visuelle. L'avancement s'est découpé en trois phases majeures (Sprints) suivant la méthode Agile Scrum.

A) Sprint 1 : Étude, Documentation et Mise en place technique (La genèse du pipeline)

Détail d'un ticket technique (Issue) pour la création de mon premier flux Kestra :

POC BI - Kestra - Création du flow centreon host-template

Edit

Closed Issue created 4 weeks ago by **Julien CHEMINADE**

Action : Développer un nouveau flow dans kestra pour listé tous les template d'hotes

Description :

Prendre en exemple le flow Kestra qui permet de listé tous les hôtes présent dans centréon et créer un nouveau flow pour listé l'ensemble des templates de host.

lien vers le flow d'exemple : http://liv1e25.alt:8080/ui/main/flows/edit/company.infra/centreon_raw_host

lien vers la documentation de l'api centreon : https://docs-api.centreon.com/api/centreon-web/24.10/#tag/Host-template/paths/\~1configuration\~1hosts\~1templates/get

Url de l'api pour récupérer les template d'hôte à utilisé dans le flow :

```
"{{ inputs.centreon_url }}/configuration/hosts/templates"
```

Re utilisé la partie inputs (variables) et login du flow d'exemple car standard à tous les futurs flow centreon :

```
inputs:
  - id: centreon_url
    type: STRING
    displayName: URL Centreon
    defaults: "http://monitoring-dev.alturing.eu/centreon/api/latest" # Remplace par ton IP

  - id: search_limit
    type: INT
    displayName: API Object seach limit
    defaults: 10000

tasks:
  # Connect to centreon api
  - id: login
    type: io.kestra.plugin.core.http.Request
    method: POST
    uri: "{{ inputs.centreon_url }}/login"
    contentType: application/json
    body: |
      {
        "security": {
          "credentials": {
            "login": "{{ secret('CENTREON_USER') }}",
            "password": "{{ secret('CENTREON_PASSWORD') }}"
          }
        }
      }
```

Avant de pouvoir faire circuler la donnée, la première étape a consisté à préparer l'environnement technique et à choisir les bons outils. J'ai d'abord réalisé un travail d'étude en rédigeant la documentation d'installation pour Data Build Tools (dbt), l'outil prévu pour la transformation, ainsi que pour Airbyte. Face aux instabilités d'Airbyte, l'équipe a pris la décision technique de pivoter vers Kestra pour l'orchestration des données. Pour m'adapter à ce nouvel outil, j'ai développé mon tout premier flux d'extraction de test (le flux host-template). Cette étape a été cruciale car elle m'a permis de valider la connexion à l'API Centreon en récupérant les accès de manière sécurisée via le Vault Alturing, le gestionnaire de secrets de l'entreprise.

B) Sprint 2 : Mise en place des flux de données et stockage (L'extraction massive)

Résultat d'une extraction Kestra formatée pour la base de données :

name	alias	id	log_date
All	All	1	2026-02-09T13:27:23.731666Z
ALT	ALT	2	2026-02-09T13:27:23.731666Z
ALT-Part-IS-senonches	ALT-Part-IS-senonches	747	2026-02-09T13:27:23.731666Z
ALT-Partenaire-IS	ALT-Partenaire-IS	3	2026-02-09T13:27:23.731666Z
ALT-SC-CSV-Angers	ALT-SC-CSV-Angers	4	2026-02-09T13:27:23.731666Z
ALT-SC-ESX-Angers	ALT-SC-ESX-Angers	5	2026-02-09T13:27:23.731666Z
ALT-Siege	ALT-Siege	6	2026-02-09T13:27:23.731666Z
ALT-SYSTEM-ESX	ALT-SYSTEM-ESX	7	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-Epo	ALT-ZAPPLI-Epo	8	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-FootPrint	ALT-ZAPPLI-FootPrint	9	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-GLPI	ALT-ZAPPLI-GLPI	10	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-OPENPROJECT	ALT-ZAPPLI-OPENPROJECT	758	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-PortailCSV	ALT-ZAPPLI-PortailCSV	11	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-Qlikview	ALT-ZAPPLI-Qlikview	12	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-QualityC	ALT-ZAPPLI-QualityC	13	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-URL-DPD	ALT-ZAPPLI-URL-DPD	14	2026-02-09T13:27:23.731666Z
ALT-ZAPPLI-Vault	ALT-ZAPPLI-Vault	742	2026-02-09T13:27:23.731666Z

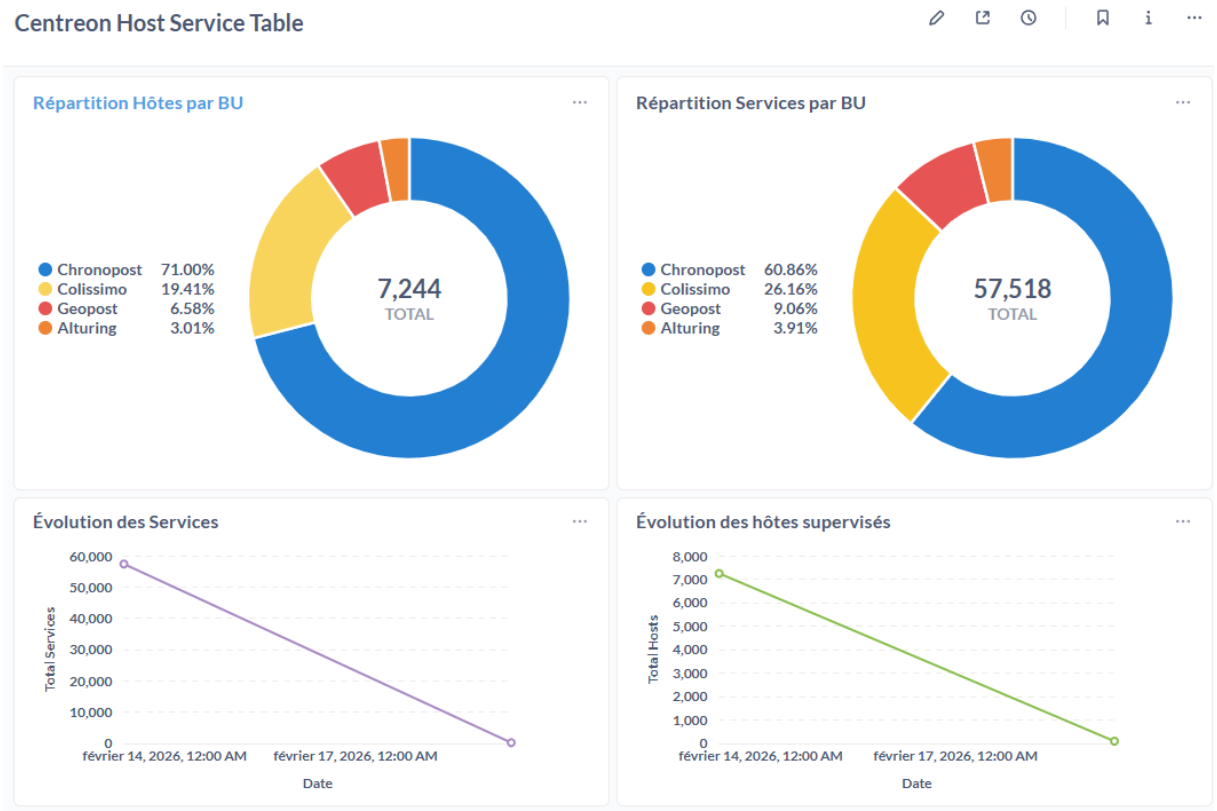
Une fois la mécanique d'orchestration validée, mon objectif a été d'ouvrir les vannes pour récupérer le maximum d'informations depuis notre source principale : Centreon. J'ai dû étudier en profondeur l'API Centreon (et contourner les limites de la V2 en basculant sur la V1) pour multiplier les flux d'extraction dans Kestra. J'ai ainsi automatisé la récupération des groupes d'hôtes (host_group), des modèles de services (service-template) et des périodes de temps (tp-template). Ensuite, il fallait réceptionner cette donnée. Comme le montre l'architecture, Kestra "pousse les données brutes issues des sources" vers la Postgres Instance. J'ai donc intégré dans Kestra la création automatique des schémas SQL pour générer les tables dans la base de données, en prenant soin d'ajouter des index pour optimiser les recherches. À la fin de ce sprint, les données brutes arrivaient correctement dans la couche "Bronze" de notre base de données.

C) Sprint 3 : Finalisation des flux et début du dashboarding

Requête SQL rédigée dans Metabase pour contourner l'absence temporaire de dbt :

La dernière phase de mon stage a consisté à finaliser l'acheminement des données et à prouver leur utilité pour les utilisateurs finaux. Après avoir clôturé les derniers flux spécifiques dans Kestra (comme les liens d'actions url_action), je suis passé à l'étape de la restitution. Dans l'architecture nominale, dbt devait nettoyer les données vers la couche Silver, puis les préparer pour la couche Gold. Ce module dbt n'étant pas encore pleinement déployé en production durant mon stage, j'ai fait preuve d'adaptabilité. Pour livrer un résultat concret, je me suis connecté directement à Metabase. J'y ai rédigé des requêtes SQL complexes pour effectuer moi-même le tri et l'agrégation des données. C'est ce qui m'a permis de générer les premiers tableaux de bord dynamiques (répartition par BU, évolution temporelle), finalisant ainsi le cycle de la donnée jusqu'à sa visualisation par les équipes métiers.

Exemples de tableaux de bord finaux livrés pour le pilotage de l'infrastructure :



V) CONCLUSION

Ce stage de 6 semaines chez Alturing a été une expérience particulièrement formatrice et enrichissante, tant sur le plan technique que professionnel. J'ai pu découvrir concrètement le fonctionnement d'une ESN de taille importante et comprendre les enjeux critiques liés à la supervision et à la traçabilité des flux logistiques dans un contexte industriel.

Sur le plan technique, la mise en place de cette Modern Data Stack m'a permis de maîtriser des outils professionnels d'orchestration (Kestra), de gestion de données (PostgreSQL) et de visualisation (Metabase). J'ai particulièrement apprécié l'approche "workflow as code" qui permet de versionner et d'automatiser entièrement les pipelines de données, une compétence clé dans le monde DevOps moderne.

L'adoption de la méthode Agile Scrum m'a également permis de travailler en mode projet réel, avec des sprints, des réunions de suivi et une gestion fine des tickets via GitLab. Cette expérience m'a conforté dans mon choix de spécialisation en développement logiciel et cybersécurité.

Pour conclure, ce stage a parfaitement complété ma formation BTS SIO SLAM en couvrant des compétences essentielles comme la gestion de projet, la documentation technique et le travail en équipe.